Models and Tools for Cyber-Physical Systems

Digital Written Exam, June the 10th 2024, 10:00-14:00

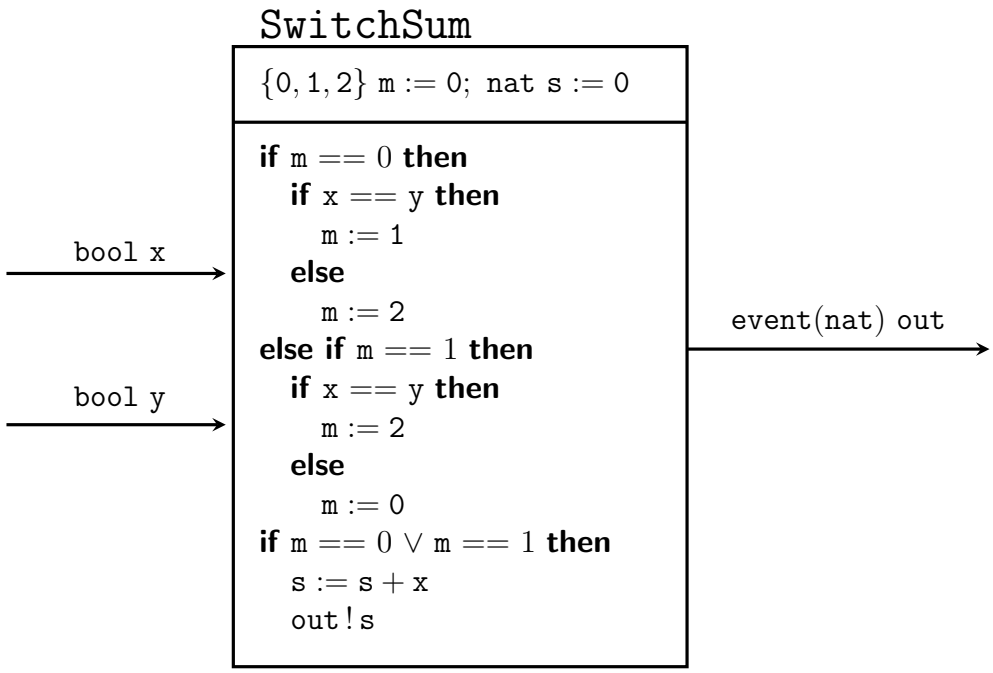
Please read the following before solving the exercises.

- This exam contains 6 exercises, worth 75 points in total. Each exercise is compulsory. The solution has to be composed in English. If you believe that the assignment wording is ambiguous or erroneous, then write down what additional assumption you are using and outline your reasons.
- In some exercises you are asked to extend incomplete UPPAAL and Python files. Solutions have to be uploaded to Digitaleksamen in the form of UPPAAL, Python, pdf or text files. For each of your solutions, ensure that it is clear which exercise is solved.
- The exam is open book, but asking/receiving help from others is not allowed.
- In case of emergencies: students can contact the instructors during the exam by approaching the study secretary.

Last but not least, good luck!

Exercise 1: Synchronous Components 10 Points

Consider the synchronous component `SwitchSum` given as a block diagram below.



Hint: Recall that `out ! s` is our notation for the event assignment `out` := `s`, and that $\perp$ denotes that an event is not present.

(a) Below we wrote the initial state and a sequence of inputs. Fill in the blanks (i.e., both the missing states between the arrows and the missing outputs on the arrows) to obtain a proper prefix of a corresponding execution.

$$(0,0) \xrightarrow{(0,0)/} \underline{\qquad\qquad} \xrightarrow{(0,1)/} \underline{\qquad\qquad}$$

$$\xrightarrow{(1,1)/} \underline{\qquad\qquad} \xrightarrow{(1,0)/} \underline{\qquad\qquad}$$

(b) Give a <u>sequence of inputs</u> leading to the state $(2,2)$.
(You <u>do not</u> have to provide the intermediate states and outputs.)

(c) Describe the component `SwitchSum` as an extended state machine.
Use the mode of the state machine to capture the value of the state variable `m`, while the state variable `s` should be updated using assignments in the mode switches.

.....Solution sketch .....

(a) $(0,0) \xrightarrow{(0,0)/0} (1,0) \xrightarrow{(0,1)/0} (0,0) \xrightarrow{(1,1)/1} (1,1) \xrightarrow{(1,0)/2} (0,2)$

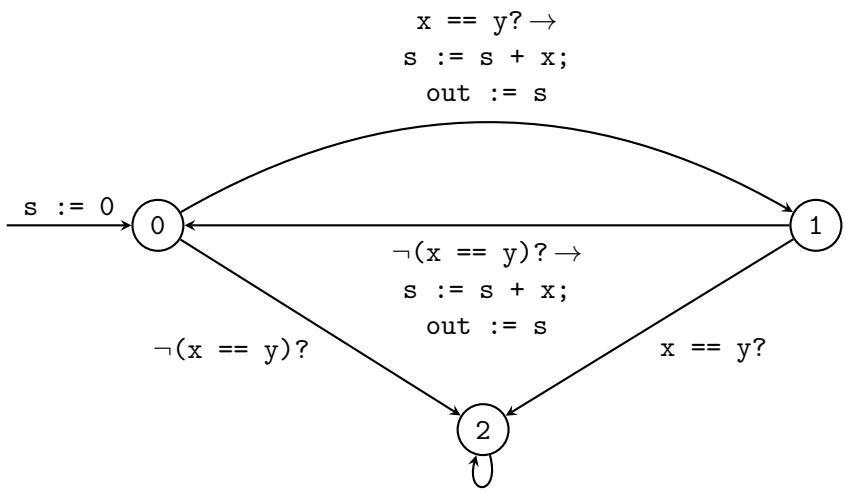
(b) The shortest solutions have length three. Below are all valid length-three input sequences.

$$(1,1),(1,0),(0,1) \qquad (1,1),(1,0),(1,0)$$

Executions corresponding to these sequences (only differing in the last input):

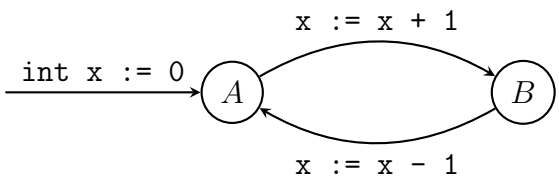
$$(0,0) \xrightarrow{(1,1)/1} (1,1) \xrightarrow{(1,0)/2} (0,2) \xrightarrow{(0,1)/\bot} (2,2)$$
$$(0,0) \xrightarrow{(1,1)/1} (1,1) \xrightarrow{(1,0)/2} (0,2) \xrightarrow{(1,0)/\bot} (2,2)$$

(c) Note the self-loop in mode 2, which is required to let the executions continue.



Exercise 2: Reachable States 10 Points

Consider the following extended state machine.



Let φ_1 be the formula $x = 0 \vee x = 1$.

(a) Provide the reachable part of the corresponding transition system.
Hint: Remember that the mode is an implicit state variable.

(b) Is φ_1 an invariant? Shortly justify your answer.

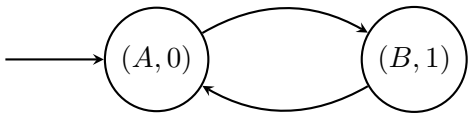
(c) Is φ_1 an inductive invariant? Shortly justify your answer.

(d) Provide another formula φ_2 different from φ_1 that is an invariant.
(You do not have to justify why φ_2 is an invariant.)

(e) Provide another formula φ_3 different from φ_1 and φ_2 that is an inductive invariant.
(You do not have to justify why φ_3 is an inductive invariant.)

.....Solution sketch .....

(a) We use the state order $(\texttt{mode}, \texttt{x})$.



(b) Yes, because all reachable states satisfy φ_1 (which is the definition of an invariant).

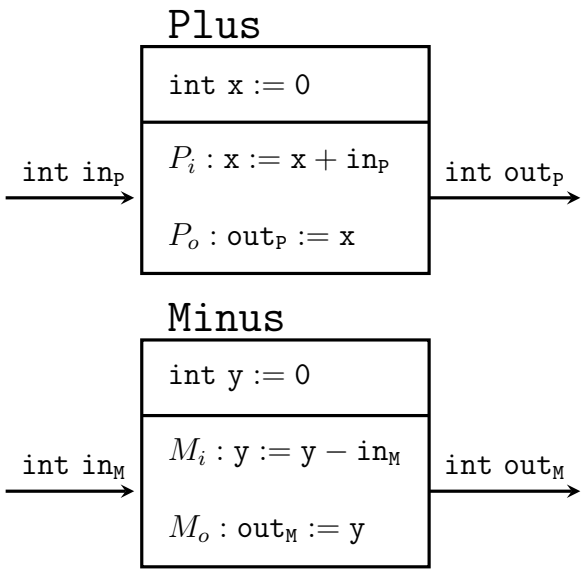
(c) No, because the state $(A, 1)$ satisfies φ_1, but one of its successor states $(B, 2)$ does not.

(d) An example is $\texttt{x} \geq 0$.

(e) An example is $(\texttt{mode} = A \wedge \texttt{x} = 0) \vee (\texttt{mode} = B \wedge \texttt{x} = 1)$.

Exercise 3: Asynchronous Components 10 Points

Consider the asynchronous components `Plus` and `Minus` given as block diagrams below.



Consider the parallel composition given by the following description:

$$\texttt{Plus}[\texttt{out}_\texttt{P} \mapsto \texttt{temp}] \quad | \quad \texttt{Minus}[\texttt{in}_\texttt{M} \mapsto \texttt{temp}] \quad \backslash \quad \texttt{temp}$$

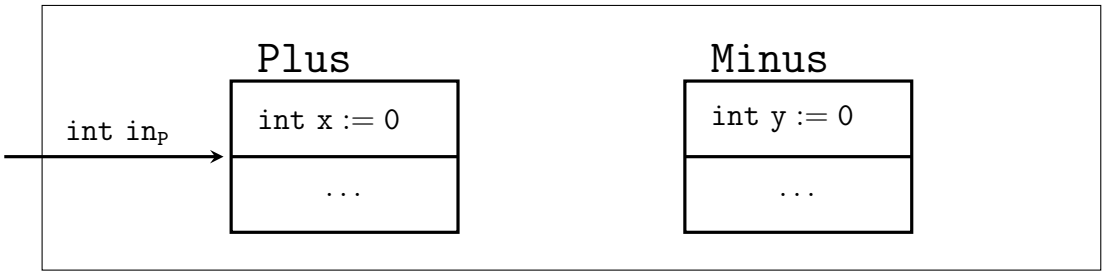
(a) Below is a partial block diagram for the parallel composition. Connect the inputs and outputs of the components correctly with edges/arrows. Inputs and outputs of the composition should leave the outer block; we have given an example for the input $\texttt{in}_\texttt{P}$.

(b) Annotate the edges you drew with the channel types and names.

(c) Give all input tasks, internal tasks, and output tasks of the parallel composition.

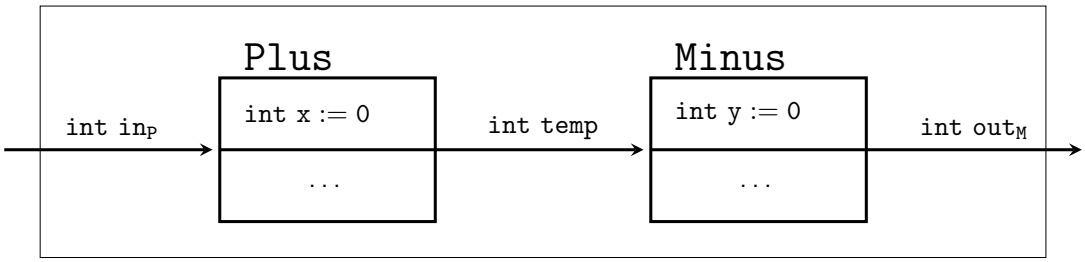
(d) Give an execution for the input sequence $-1\ 2\ -1$ and output sequence $1\ 0$.

Hint: The local variable `temp` is not part of the state. Also, remember that executions for asynchronous components perform exactly one task in each step. Input tasks have the form $s \xrightarrow{\texttt{i}?v} s'$, internal tasks have the form $s \xrightarrow{\varepsilon} s'$, and output tasks have the form $s \xrightarrow{\texttt{o}!v} s'$, where v is a value and `i` and `o` are input and output variables, respectively.



Solution sketch

(a) and (b)



(c)

input: $\mathtt{x} := \mathtt{x} + \mathtt{in_P} \quad (= P_i)$

internal: $\mathtt{temp} := \mathtt{x};\ \mathtt{y} := \mathtt{y} - \mathtt{temp}$ (or shortened: $\mathtt{y} := \mathtt{y}$ - $\mathtt{x}$)

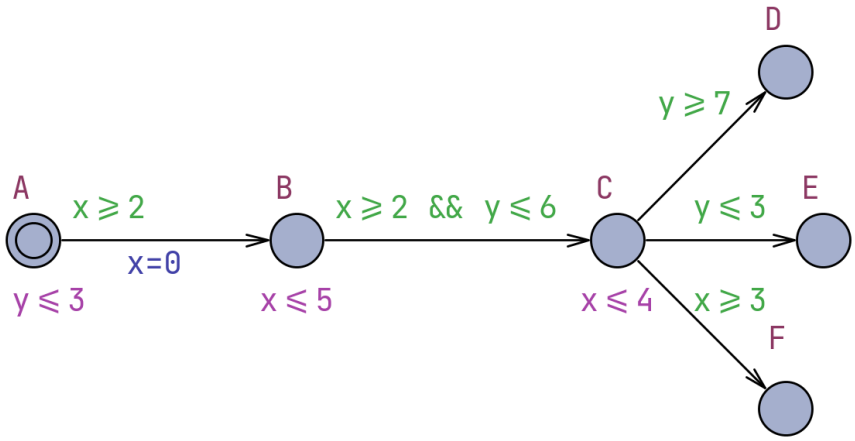
output: $\mathtt{out_M} := \mathtt{y} \quad (= M_o)$

(d) We use the state order $(\mathtt{x}, \mathtt{y})$. The shortest execution is (line break for space reasons):

$$(0,0) \xrightarrow{\mathtt{in_P}\,?\,-1} (-1,0) \xrightarrow{\varepsilon} (-1,1) \xrightarrow{\mathtt{out_M}\,!\,1} (-1,1) \xrightarrow{\mathtt{in_P}\,?\,2} (1,1) \xrightarrow{\varepsilon} (1,0)$$
$$\xrightarrow{\mathtt{out_M}\,!\,0} (1,0) \xrightarrow{\mathtt{in_P}\,?\,-1} (0,0)$$

Exercise 4: Exploration of Timed Automata 10 Points

Consider the timed automaton $\mathcal{A}$ below with two clocks x and y.



(a) Which of the three locations D, E and F are reachable from the initial state (A, $\mathtt{x} = 0, \mathtt{y} = 0$)?

(b) For each of these three goal locations that is reachable, provide a timed transition sequence that leads to the location from the initial state.

(c) For each of the three goal locations that is reachable, what is the fastest time of reaching that location. Provide a witness timed transition sequence.

(d) Describe, using difference constraints or its graphical representation, the reachable zones upon entry and after delay for the locations A, B and C.

(e) For each of the three goal locations that is NOT reachable, investigate whether it is possible to weaken the guard leading to the location, so that the location becomes reachable.
NOTE: `x<=7` is weaker than `x<=5`. Similarly, `x>=2` is weaker than `x>=4`.

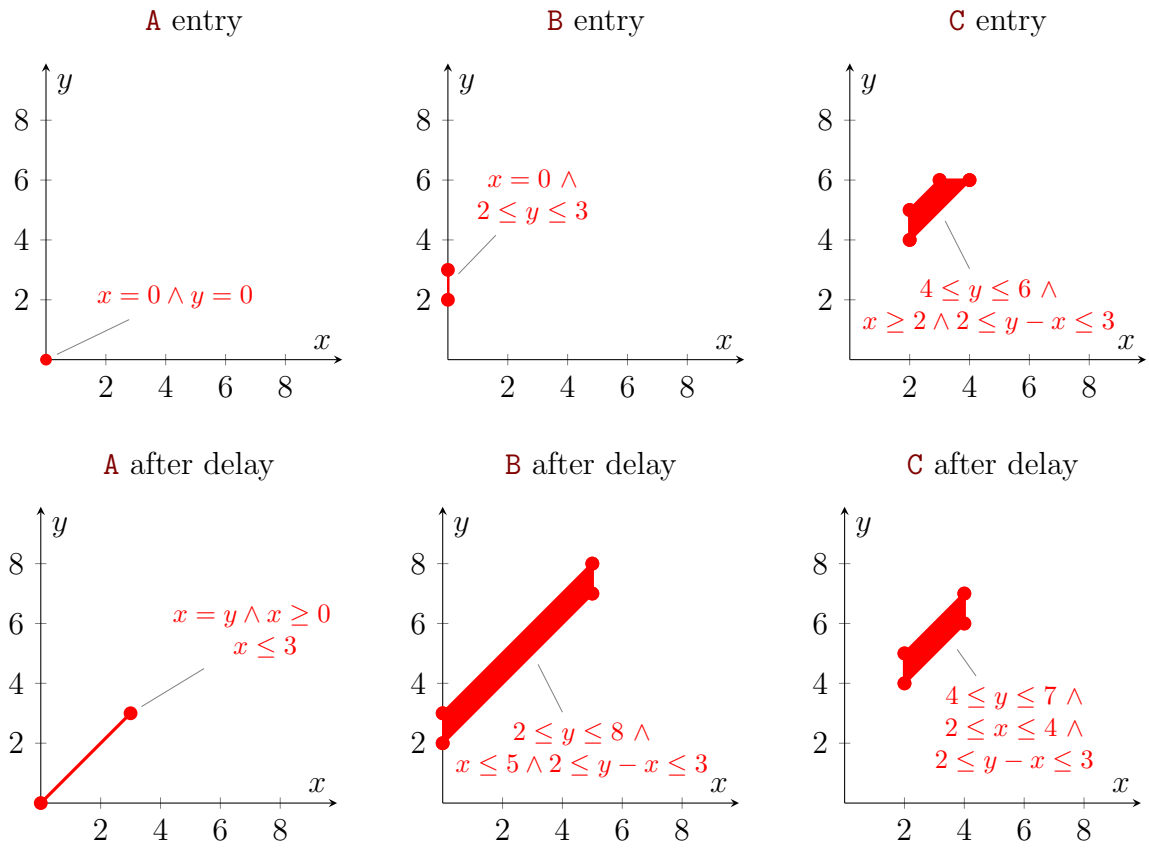
Solution sketch

(a) Both D and F are reachable.

(b) $(\mathtt{A}, x = 0, y = 0) \xrightarrow{3} (\mathtt{A}, x = 3, y = 3) \rightarrow (\mathtt{B}, x = 0, y = 3) \xrightarrow{3} (\mathtt{B}, x = 3, y = 6) \rightarrow (\mathtt{C}, x = 3, y = 6) \xrightarrow{1} (\mathtt{C}, x = 4, y = 7) \rightarrow (\mathtt{D}, x = 4, y = 7)$
and
$(\mathtt{A}, x = 0, y = 0) \xrightarrow{2} (\mathtt{A}, x = 2, y = 2) \rightarrow (\mathtt{B}, x = 0, y = 2) \xrightarrow{3} (\mathtt{B}, x = 3, y = 5) \rightarrow (\mathtt{C}, x = 3, y = 5) \rightarrow (\mathtt{F}, x = 3, y = 5)$

(c) The fastest time to reach D is 7 time units, and the fastest time to reach F is 5 time units. The witnessing sequences are above.

(d) The reachable zones upon entry and after delay:



(e) Changing the guard to $y \leq 4$ will make E reachable.

Exercise 5: Hammer game 15 Points

In this exercise we consider the hammer game. In this game, people stand around a wooden block and need to hit their own three nails into the block. But there is only a single hammer for hitting the nails. Each time the hammer is placed on the block, the participants rush to grab the hammer so they can hit one of their nails. After hitting a nail, the hammer is placed back and the round repeats. The goal is to be the first one having your three nails hit into the block.
In this instance of the game, Christian, Kim, Martijn, and Max are playing. Each person has different reaction times, also depending on the progress of the game. The reaction times are indicated in the table below, where each cell indicates the lower and upper bound of each action by each player. For example, trying to grab the hammer for the second nail takes Kim between 6 and 12 time units.

Reaction and hitting times	Christian	Kim	Martijn	Max
Grabbing hammer 1	$[2, 15]$	$[7, 10]$	$[4, 11]$	$[7, 12]$
Hitting nail 1	$[4, 13]$	$[6, 11]$	$[5, 12]$	$[5, 12]$
Grabbing hammer 2	$[3, 13]$	$[6, 12]$	$[7, 11]$	$[2, 10]$
Hitting nail 2	$[5, 12]$	$[5, 13]$	$[7, 15]$	$[4, 12]$
Grabbing hammer 3	$[6, 13]$	$[4, 12]$	$[6, 14]$	$[6, 15]$
Hitting nail 3	$[7, 15]$	$[3, 15]$	$[3, 10]$	$[8, 14]$

Now the key question is how this game will progress with these players. How long are they playing? Is this a fair game and has everyone the opportunity to win? Or is there always one player who wins?
Below you can find the screenshot of the UPPAAL model of the problem. Here each player is modelled as a timed automaton where the horizontal sequence of transitions reflects grabbing and subsequently hitting the three nails. The selfloops at each `Ready` location indicates misgrabbing the hammer. In that case, the player has to wait until the hammer is back on the block. The minimum and maximum reaction times are modelled using constraints on a local clock. The fact that the hammer can at most be taken by one person is modelled using broadcast channels, where a broadcast channel is a one-to-many channel (for example, if one player picks the hammer with `pick!`, *all* the other players will go to `Waiting` with `pick?`). You can find the complete UPPAAL model in Digital Exam in the file `HammerGame.xml`. (For the first part, you can ignore the templates `Cheater` and `LatePlayer`.)

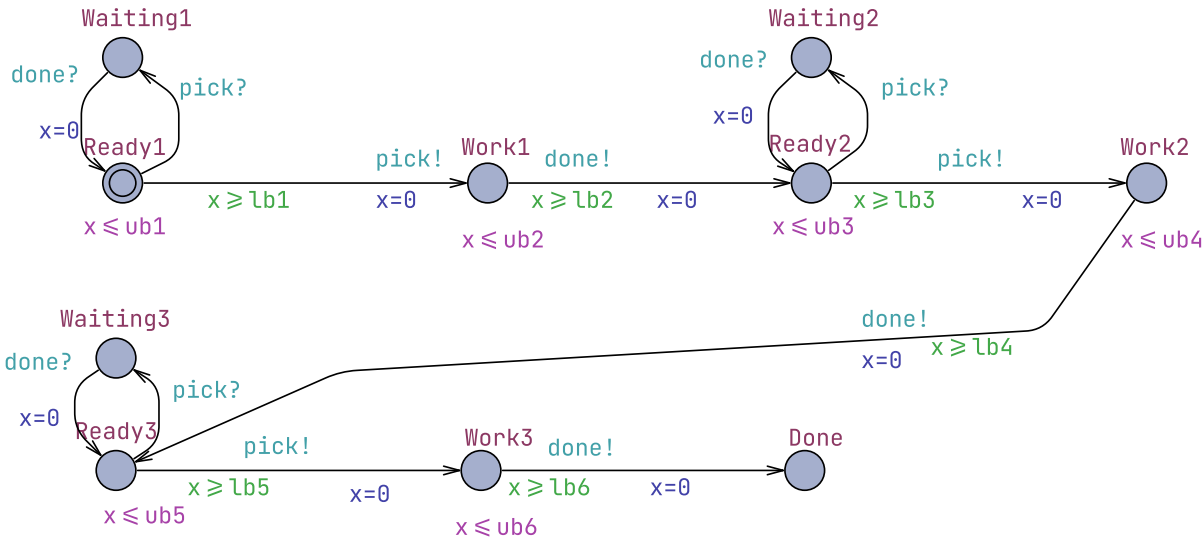


Figure 1: UPPAAL model of a player in the hammer game

Now answer the following questions:

(a) Formulate as a UPPAAL query ϕ_1 that Christian can finish (i.e., reaching the `Done` location) within 30 time units. Check in UPPAAL whether ϕ_1 is satisfied, and if so, provide a schedule indicating Christian's strategy. You can illustrate the schedule using an Message Sequence Chart (MSC) chart for ϕ_1.

Hint: In the symbolic simulator of UPPAAL, you can save the generated MSC by right-clicking it and select "Export".

(b) Use UPPAAL to find the minimum completion time, i.e., the minimum time needed until all players are finished. Describe how you obtained this number.

(c) Formulate as a UPPAAL query ϕ_2 that Christian is able to win the game (i.e., being the first player to be done). Check in UPPAAL whether ϕ_2 is satisfied. Subsequently, check in UPPAAL whether the game is fair (each player is able to win).

(d) Formulate as a UPPAAL query ϕ_3 that always after playing the game for 300 time units the game is completed, i.e., all 4 players are finished. Check in UPPAAL whether ϕ_3 is satisfied.

In the following modify the model in `HammerGame.xml` in three different (independent!) ways.

(e) Martijn has been drinking a bit too much beer during the game, so all his reaction times are 10 time units slower, i.e., $[4, 11]$ becomes $[14, 21]$, $[5, 12]$ becomes $[15, 22]$, and so on. Modify the original model `HammerGame.xml` accordingly. Is it still a fair game? And what is the minimum completion time of the game?

Please upload to Digital Exam your modified UPPAAL model in a file with name `HammerGame2.xml`.

(f) In a rematch of the game, Christian and Max decided to cheat in the game. The first time one of them has a hammer, the cheater hits both the first and the second nail. Modify the original model `HammerGame.xml` accordingly. You can use the UPPAAL template `Cheater` to model this. Is it still a fair game? And what is the minimum completion time of the game?

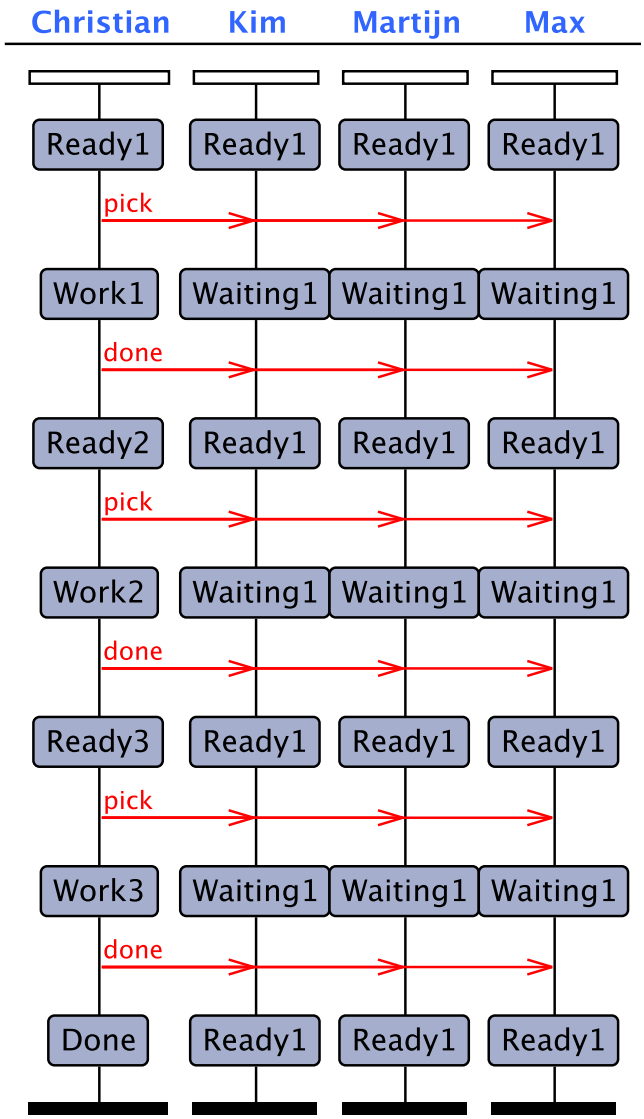
Please upload to Digital Exam your modified UPPAAL model in a file with name `HammerGame3.xml`.

(g) In a rematch of the game, Kim arrived too late, as he was busy working on a UPPAAL model. He arrived when all other players already have hit their first nail. Modify the original model `HammerGame.xml` accordingly. You can use the UPPAAL template `LatePlayer` to model this. Is it still a fair game? And what is the minimum completion time of the game?

Please upload to Digital Exam your modified UPPAAL model in a file with name `HammerGame4.xml`.

. . . . . Solution sketch . . . . . .

(a) ϕ_1: `E<> Christian.Done and time<=30`. The query is satisfied. The Message Sequence Chart of an optimal schedule (which also satisfies ϕ_1) is shown below.



(b) The minimum completion time is 122 time units. One can obtain this by doing a binary search on the lower bound of the time using the query `E<> time<=122 and Christian.Done and Kim.Done and Martijn.Done and Max.Done`, or one can ask UPPAAL to generate the fastest diagnostic trace for `E<> Christian.Done and Kim.Done and Martijn.Done and Max.Done` and inspecting the Symbolic Simulator for the value of `time`.

(c) ϕ_2: `E<> Christian.Done and not Kim.Done and not Martijn.Done and not Max.Done`. The query is satisfied. The three similar queries for the other players show that every player is able to win, so the game is fair.

(d) ϕ_3: `A[] time>=300 imply Christian.Done and Kim.Done and Martijn.Done and Max.Done`, or alternatively ϕ_3: `A<> Christian.Done and Kim.Done and Martijn.Done and Max.Done and time<=300`. The query is not satisfied.

(e) The game is no longer fair, as `E<> Martijn.Done and not Christian.Done and not Kim.Done and not Max.Done` is no longer satisfied. The new minimum completion time is 182.

(f) One could model a cheater by redirecting the `done!` from `Work1` to `Ready3` (interpretation 1) or to `Work2` (interpretation 2, without `done!` on this new transition). With both interpretations of the question, the game is still fair. The minimum completion time is 108 time units (interpretation 1) or 117 time units (interpretation 2).

(g) One way of modeling this situation is by including a new initial location in which a late player waits. A new global variable `nails` counts the number of first nails hit by regular players. Only if this is 3 or higher can a late player join the game, i.e., transition to `Waiting1`. The game is still fair, as Kim is still able to win. The minimum completion time is 122 time units.

Exercise 6: Windy Car 20 Points

Consider the model of a car that is moving along the x-axis from left to right. While moving, the car is exposed to wind along the y-axis (from up and down). Assuming that the mass of the car is $m = 1000$ (kg) and that the friction coefficient is $k = 50$, this yields the following differential equations.

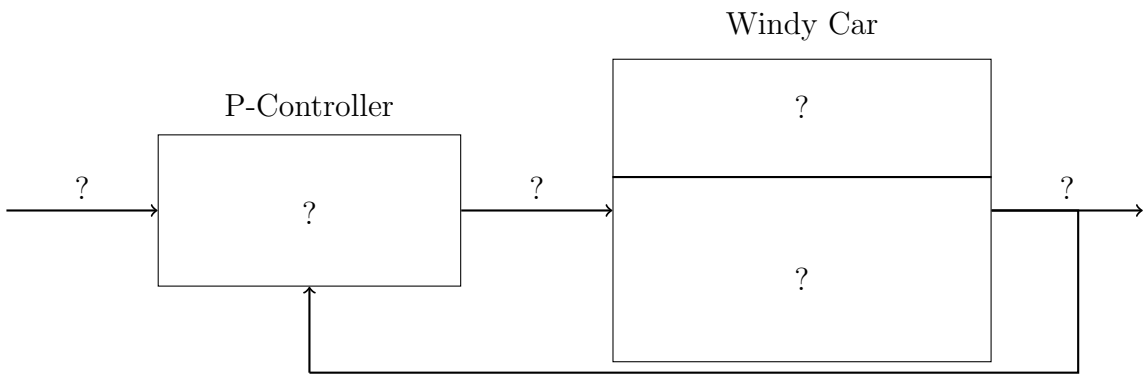
$$\frac{d}{dt}x(t) = v_x(t) \qquad \frac{d}{dt}y(t) = v_y(t)$$

$$\frac{d}{dt}v_x(t) = -\frac{k}{m}v_x(t) + \frac{1}{m}F_x \qquad \frac{d}{dt}v_y(t) = -\frac{k}{m}v_y(t) + \frac{1}{m}F_y + \frac{100}{m}\sin\left(\frac{t}{10}\right)$$

In the above equations, the sin term models vertical wind. Overall, we wish to keep the car at horizontal speed $v_x \approx 20$ and at the center of the road $y \approx 0$.

(a) Compute the horizontal force F_x which needs to be applied to achieve a steady-state velocity of $v_x \approx 20$. [If you cannot solve this exercise, assume $F_x = 100$ in the following exercises.]

(b) Measuring y, we wish to compute the vertical force F_y using a P-controller (that ensures that $y \approx 0$). Complete the block diagram below realizing this.



For the following three questions, you will need to complete the provided Python script `NumericalSolverExam24.py`. You can find the Python script in Digital Exam. After finishing this exercise, please upload your modified Python script of the windy car in a file with name `WindyCar24.py`, also in case you could only solve question c).

(c) Implement the `f` method in the provided Python script so that the overall program simulates a P-controller. Afterwards, tune the proportional gain K_p so that the car keeps within 5cm of the road center, i.e., $|y| \leq 0.05$. Describe the quality of your controller.

(d) Extend the P-controller from the foregoing exercise to a PID-controller. Extend the method `f` in the Python script accordingly.

After tuning the gains, assess the quality of the PID controller.

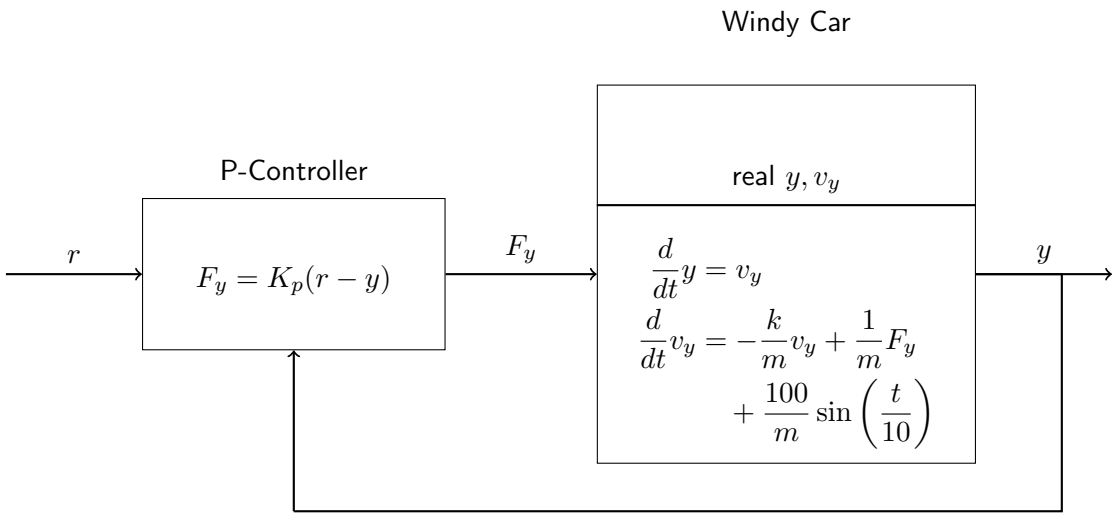
(e) Explain why the PID controller does not provide a perfect system response where y is approaching zero.

(f) Provide a controller (which is not PID) achieving $y(t) = 0$ for all $t \geq 0$.

.....Solution sketch .....

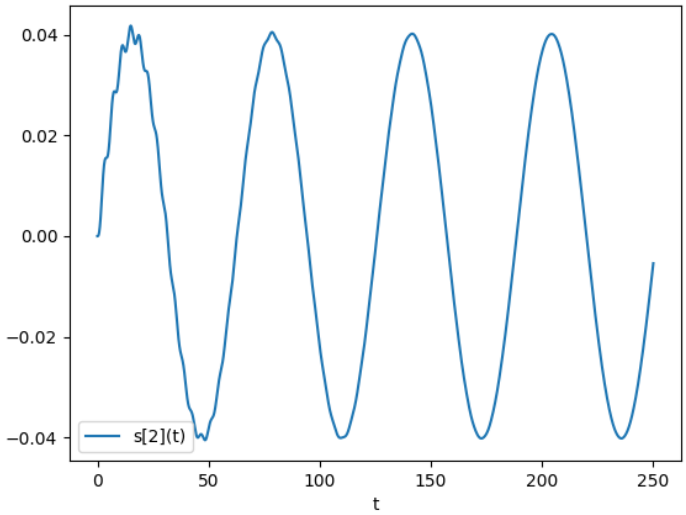
(a) Steady-state velocity implies that the velocity does not change, which translates to $\frac{d}{dt}v_x(t) = 0$. Using the given equation and some rewriting, we could obtain $F_x = kv_x = 50 \cdot 20 = 1000$.

(b) The block diagram is completed as follows. Note that the dynamics in the x-dimension is fully independent of the y-direction, hence we excluded those state variables and equations from the block diagram below as they don't contribute to the construction of the P-controller.



(c) For formulas and code, please see d). The figure below shows the y-position of the car, and is obtained with $K_p = 2500$. After some initialization (roughly until $t = 100$), an maximum deviation of 0.05 cm is achieved. This is a nice performance compared to the potential deviation of 30m for the car without any control ($K_p = 0$). The initialization phase will be uncomfortable for a drive, due to the small oscillations (higher frequencies on the signal). For this case, the higher the proportional gain, the smaller the maximum errors, but also the stronger the higher-order oscillations are and the longer this initialization phase last.

Oscillation errors can be in principle addressed by extending the P-controller to a PD-controller.

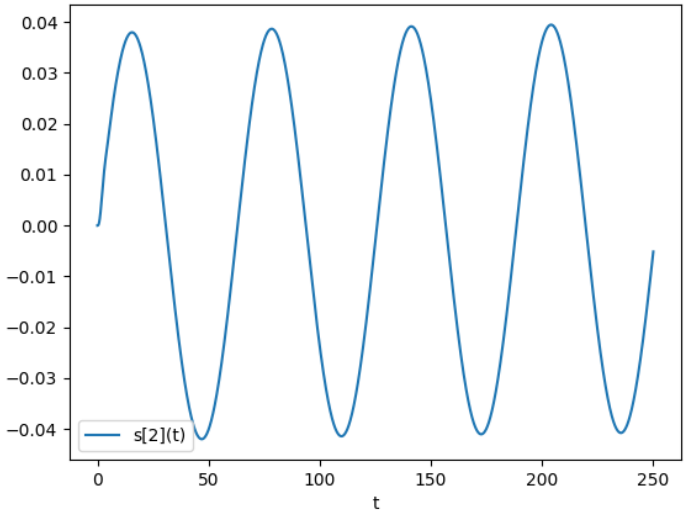


(d) The derivative error is equivalent to $\frac{d}{dt}e(t) = \frac{d}{dt}(r - y(t)) = -v_y(t)$. With this, we can implement the derivative part of the PID-controller. Furthermore, instead of being given by $I(t) = \int e(t)\mathrm{d}t$, the integral error enjoys the differential equation $\frac{d}{dt}I(t) = e(t) = r - y(t)$. This results in an additional state variable (so we need to adapt the initial state declaration in `main` as well!), which we can use to define the integral part of the PID-controller. (Note that the PD-controller is obtained by letting $K_i = 0$, while the P-controller if $K_i = K_d = 0$.)

The figure below shows the y-position of the car, and is obtained with $K_p = 2500$, $K_d = 1000$, and $K_i = 15$. Trying different gains, it can be noted that the D-part is only able to remove the high-frequency oscillations during the initialization phase; it fails to remove the low-frequency oscillations related to the wind. With other words, the controlled car is stable around $y = 0$, but not asymptotically stable. (We also note that the integral part is not strictly necessary, as the y-position already moves around the reference signal. But note that high values of the integral gain will make the system unstable.)

```
def f(x, t):
  # Implements the car model including proportional and derivative controller
  m = 1000
  k = 50

  Kp = 2500
  Kd = 1000
  Ki = 15
  Fx = 1000 # Constant force to horizontal speed ensure movement, 100 if a) is left unsolved
  r = 0

  # x=x[0], vx=x[1], y=x[2], vy=x[3], int (0-y(t)) dt = x[4]
  Fy = Kp*(r - x[2]) + Kd*(0 - x[3]) + Ki*x[4]
  der_x = x[1]
  der_y = x[3]
  der_vx = -(k/m)*x[1] + Fx/m
  der_vy = -(k/m)*x[3] + Fy/m + 100*(math.sin(t/10)/m)
  der_I = -x[2]
```

```python

  return [der_x, der_vx, der_y, der_vy, der_I]

def main():
  x0 = [0, 20, 0, 0]  # Initial state for answer c), as provided in given script.
  x0 = [0, 20, 0, 0, 0]  # Initial state for answer d.
  ...
```



(e) A PID-controller is a feedback controller, i.e., it reacts to the error signal. An error needs to be created before a PID-controller starts to provide some non-zero input to the system. Since the error in this case is caused by a continuously changing disturbance (the wind) with mean zero, the PID-controller will always be behind this disturbance.

(f) From the previous answer, we have learned that reactive control is insufficient. Pro-active control, i.e., anticipating errors, can actually achieve the desired result, which is called feed-forward. The perfect control would be $F_y(t) = -100 \sin\left(\frac{t}{10}\right)$, the precise inverse of the wind.

One could also derive this mathematically using the given equations as follows. We want no change in y, so $\frac{d}{dt}y(t) = v_y(t) = 0$. To keep the velocity in the y-direction at zero for all $t \geq 0$, we need $\frac{d}{dt}v_y(t) = -\frac{k}{m}v_y(t) + \frac{1}{m}F_y + \frac{100}{m}\sin\left(\frac{t}{10}\right) = 0$ for all $t \geq 0$. Substituting $v_y(t) = 0$ in this equation and rearranging the remaining parts results in the perfect input provided above.

Note that this control is too perfect though; indeed, it is in some sense prophetic because it requires to know the precise (future) wind disturbance.